

Apprentissage Adaptatif et Continu pour la Reconnaissance d'Activité Humaine

Explication complète du projet — de zéro à la présentation

PFE — École Nationale Supérieure d'Informatique (ESI)

Année universitaire 2025/2026

Étudiante : NEDJAM Walaa

Encadrante : Attal Ferhat (LISSI, UPEC Paris)

Co-encadrante : MEZIANI Lila

19 mai 2026

Table des matières

1	Le problème de départ	3
1.1	C'est quoi la reconnaissance d'activité humaine (HAR)?	3
1.2	Pourquoi c'est utile?	3
1.3	Le vrai problème : le modèle oublie	3
2	Notre solution en 3 niveaux	4
3	Les données utilisées	4
3.1	D'où viennent les données?	4
3.2	Problème : les données sont hétérogènes	4
3.3	Notre pipeline de prétraitement (3 étapes)	5
4	Le backbone : l'encodeur Transformer	5
4.1	C'est quoi un Transformer?	5
4.2	Comment fonctionne-t-il?	5
4.3	Paramètres du backbone	6
5	Module 1 : Reconnaissance continue	6
5.1	Le problème spécifique	6
5.2	Solution 1 : La mémoire des prototypes	6
5.3	Solution 2 : Le tampon de rejeu (Experience Replay)	7
5.4	Solution 3 : La perte contrastive supervisée	7
5.5	La fonction de perte totale	7
6	Module 2 : Anticipation d'activité	7
6.1	Le problème	7
6.2	Architecture	8
6.3	Entraînement	8
7	L'entraînement en deux phases	8
7.1	Phase 1 : Pré-entraînement supervisé	8
7.2	Phase 2 : Apprentissage continu	8
8	Les métriques d'évaluation	8
9	Résultats expérimentaux	9
9.1	Pré-entraînement	9
9.2	Scénario utilisateur-incrémental (44 tâches)	10
9.3	Scénario classe-incrémental (6 tâches)	10
9.4	Anticipation d'activité	11
10	Contribution originale Replay pondéré par incertitude	12
10.1	Idée	12

10.2 Intuition	13
10.3 Paramètre α	13
11 Visualisation — Ce que le modèle a appris	13
11.1 t-SNE des embeddings	14
11.2 Poids d'attention du Transformer	14
12 Ablation study — Chaque composant justifié	15
13 Pourquoi certains choix ont été faits	16
13.1 Pourquoi pas d'apprentissage auto-supervisé ?	16
13.2 Pourquoi pas de modèles de fondation pré-entraînés ?	16
14 Résumé général	17
14.1 Perspectives et travaux futurs	18

Le problème de départ

C'est quoi la reconnaissance d'activité humaine (HAR) ?

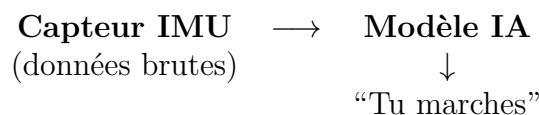
Imagine que tu portes une montre connectée ou un téléphone dans ta poche. À l'intérieur se trouve un capteur appelé **IMU** (Inertial Measurement Unit) qui contient :

- un **accéléromètre** — mesure l'accélération (mouvements linéaires)
- un **gyroscope** — mesure la rotation

Ce capteur enregistre tes mouvements **50 fois par seconde** (50 Hz), produisant un flux de données continues.

Objectif de la HAR

À partir de ce flux de données brutes, **deviner automatiquement ce que fait la personne** : marcher, courir, monter des escaliers, s'asseoir, tomber, faire du vélo...



Pourquoi c'est utile ?

- **Santé** : détecter les chutes chez les personnes âgées avant qu'elles ne se blessent
- **Médecine** : suivre les patients à distance (rééducation, Parkinson, Alzheimer)
- **Sport** : compter les calories, analyser la technique
- **Sécurité industrielle** : détecter les comportements anormaux des travailleurs

Le vrai problème : le modèle oublie

Un modèle classique est entraîné **une seule fois** sur des données fixes. Mais dans la réalité, le monde change :

- Un **nouvel utilisateur** arrive (ses mouvements sont différents)
- Une **nouvelle activité** doit être reconnue (ex. : vélo, yoga)
- Le **contexte change** (capteur différent, hôpital vs. stade)

Le catastrophic forgetting (oubli catastrophique)

Si tu ré-entraînes le modèle sur le nouvel utilisateur, **il oublie complètement les anciens**. C'est le *catastrophic forgetting* — un problème fondamental de l'intelligence artificielle.

Avant	Reconnaît bien Alice, Bob, Carol
Ré-entraîne sur David	
Après	Reconnaît bien David mais oublie Alice, Bob, Carol ×

C'est exactement ce problème que ce projet résout.

Notre solution en 3 niveaux

Notre système apprend de manière **continue** — il s'adapte à chaque nouvel utilisateur ou nouvelle activité *sans oublier* ce qu'il a appris avant.

Il est organisé en deux modules :

Backbone	Extracteur de caractéristiques partagé (Transformer)
Module 1	Reconnaissance continue (prototype memory + replay)
Module 2	Anticipation d'activité (prédire la prochaine activité)

Le signal IMU passe d'abord par le backbone qui le transforme en un **vecteur numérique** résumant l'activité en cours. Ce vecteur est ensuite utilisé par les deux modules.

Les données utilisées

D'où viennent les données ?

Nous utilisons **3 bases de données publiques** de capteurs IMU :

TABLE 1 – Bases de données utilisées.

Dataset	Activités	Sujets	Fréquence	Fenêtres
HAPT	Marche, escaliers, transitions	30	50 Hz	10 451
WISDM	Marche, course, vélo	36	20 Hz	36 554
PAMAP2	18 activités variées	9	100 Hz	9 856
Total	12 classes unifiées	44	50 Hz	56 861

Problème : les données sont hétérogènes

Chaque base de données a sa propre fréquence, ses propres unités et ses propres noms d'activités. Exemple : HAPT dit “walking” et WISDM dit “Walk” — c'est la même chose mais le modèle ne le sait pas.

Notre pipeline de prétraitement (3 étapes)

1. **Rééchantillonnage** — tout ramener à 50 Hz (même rythme)
2. **Suppression de la gravité** — filtrer la composante statique pour ne garder que le mouvement
3. **Fenêtrage glissant** — découper le signal en fenêtres de 3 secondes avec 50% de chevauchement

Résultat du prétraitement

56 861 fenêtres de 3 secondes, 6 canaux (acc x,y,z + gyro x,y,z), 12 classes d'activité unifiées, 44 sujets différents.

Les 12 classes unifiées sont : marcher, monter les escaliers, descendre les escaliers, assis/debout, courir, allongé, vélo, marche nordique, et 4 transitions posturales (s'asseoir, se lever, s'allonger, se redresser).

Le backbone : l'encodeur Transformer

C'est quoi un Transformer ?

Le Transformer est une architecture d'intelligence artificielle inventée en 2017 (célèbre grâce à ChatGPT). À la base il travaille sur du texte, mais ici on l'adapte pour des **séries temporelles de capteurs IMU**.

Rôle du backbone

Prendre une fenêtre de 3 secondes de données IMU (150 points $\times$ 6 canaux) et la **résumer en un seul vecteur de 128 nombres** qui capture l'essentiel de l'activité.

Comment fonctionne-t-il ?

1. **Projection** — les 6 canaux sont transformés en 128 dimensions
2. **Token CLS** — un token spécial est ajouté au début de la séquence (comme un résumé vide qui va se remplir)
3. **Encodage positionnel** — chaque position temporelle reçoit une signature unique (le modèle sait “ceci est le début / milieu / fin”)
4. **4 blocs Transformer** — chaque bloc fait de l’“attention” : il regarde toute la séquence et décide quelles parties sont importantes
5. **Sortie** — le token CLS contient maintenant le résumé complet de la fenêtre : un vecteur de 128 nombres

Analogie : imagine que tu lis une phrase. L’attention te permet de comprendre que dans “*la balle que Marie a frappée est rouge*”, “rouge” décrit “balle” et non “Marie”. Le Transformer fait pareil avec les données IMU : il comprend que le pic d’accélération au pas 23 est lié au creux de rotation au pas 18.

Paramètres du backbone

Paramètre	Valeur
Dimension d	128
Nombre de blocs L	4
Têtes d’attention H	4
Dimension FFN	256
Paramètres totaux	531 457

Module 1 : Reconnaissance continue

Le problème spécifique

Le backbone produit un vecteur de 128 nombres pour chaque fenêtre. Il faut maintenant **classer** ce vecteur dans l’une des 12 activités *sans oublier les activités déjà apprises* quand une nouvelle arrive.

Solution 1 : La mémoire des prototypes

Au lieu d’un classifieur ordinaire (softmax), on utilise une approche par **prototypes**. Chaque activité est représentée par **un seul vecteur moyen** — son prototype.

Prototype

Le prototype de “marcher” est la **moyenne** de tous les vecteurs produits par le backbone quand la personne marche. Il représente le “centre de gravité” de cette activité dans l’espace.

Classification : pour reconnaître une activité inconnue, on calcule la distance entre son vecteur et tous les prototypes. L’activité dont le prototype est le plus proche gagne.

Avantages :

- Ajouter une nouvelle activité = juste ajouter un nouveau prototype (pas besoin de tout ré-entraîner)
- Mise à jour en ligne avec la formule de la moyenne courante : aucune donnée ancienne ne doit être stockée

Solution 2 : Le tampon de rejeu (Experience Replay)

Pourquoi le replay ?

Même avec les prototypes, le backbone lui-même peut se dégrader quand on continue l'entraînement sur de nouvelles données. Les anciens patterns sont progressivement "écrasés".

Le tampon de rejeu conserve un **petit échantillon** des données passées (2000 fenêtres maximum, réparties équitablement entre les classes). À chaque étape d'entraînement, un mini-lot de ces données anciennes est **mélangé** avec les nouvelles données pour rappeler au modèle ce qu'il a appris avant.

Analogie : c'est comme réviser ses anciens cours en même temps qu'on apprend un nouveau chapitre. Sans révision, on oublie. Avec révision, on consolide.

Réservoir sampling : quand le tampon est plein et qu'une nouvelle donnée arrive, elle remplace aléatoirement une ancienne avec une probabilité calculée pour maintenir l'équilibre entre les classes.

Solution 3 : La perte contrastive supervisée

En plus de l'apprentissage classique, on ajoute une **perte contrastive** qui force le modèle à :

- **Rapprocher** les vecteurs de la même activité
- **Éloigner** les vecteurs d'activités différentes

Cela rend les prototypes plus robustes et la classification plus précise, surtout quand de nouvelles classes arrivent et risquent de brouiller les frontières entre classes existantes.

La fonction de perte totale

À chaque étape d'entraînement continu, on minimise :

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{CE}}(\text{nouvelles données})}_{\text{apprendre}} + \underbrace{\mathcal{L}_{\text{CE}}(\text{données du tampon})}_{\text{ne pas oublier}} + 0.1 \times \underbrace{\mathcal{L}_{\text{contrast}}}_{\text{séparer les classes}}$$

Module 2 : Anticipation d'activité

Le problème

Peut-on **prédire la prochaine activité** avant qu'elle ne commence, en observant seulement une partie du signal actuel ?

Exemple : si on voit les 50% premiers d'une marche, peut-on deviner que la prochaine activité sera "monter les escaliers" ?

Architecture

1. On prend les $S = 5$ dernières fenêtres partielles (chacune tronquée à $p\%$ de sa durée)
2. Chaque fenêtre passe par le backbone $\rightarrow$ 5 vecteurs de 128
3. Ces 5 vecteurs sont traités par un **LSTM à 2 couches** qui modélise la progression temporelle
4. La dernière sortie du LSTM prédit la prochaine activité

LSTM (Long Short-Term Memory) : un réseau de neurones spécialisé dans les séquences. Il “se souvient” des états précédents pour mieux prédire le suivant — idéal pour modéliser l’enchaînement des activités.

Entraînement

Le backbone est **gelé** (non modifié) pendant l’entraînement de ce module. Seul le LSTM apprend. On entraîne à trois ratios d’observation : $p \in \{25\%, 50\%, 75\%\}$.

L’entraînement en deux phases

Phase 1 : Pré-entraînement supervisé

On entraîne le backbone sur **toutes les données fusionnées** (56 861 fenêtres) avec une classification classique (softmax).

- Optimiseur : AdamW
- Planification du taux d’apprentissage : cosine annealing
- 30 époques, batch de 128 fenêtres

À la fin, les prototypes sont initialisés depuis les données d’entraînement.

Phase 2 : Apprentissage continu

Les tâches arrivent une par une. Pour chaque tâche (nouvel utilisateur ou nouvelle classe) :

1. Entraîner 10 époques avec la perte totale (CE + replay + contrastif)
2. Mettre à jour les prototypes
3. Ajouter les nouvelles données au tampon de rejou

Les métriques d’évaluation

On mesure les performances avec des métriques spécifiques à l’apprentissage continu :

TABLE 2 – Métriques d'évaluation.

Métrique	Définition	Idéal
Macro F1	Moyenne du F1 sur toutes les classes	Proche de 1
BWT	Dégradation des anciennes tâches après les nouvelles	Proche de 0
Forgetting	Chute maximale de performance par ancienne tâche	Proche de 0
FWT	Amélioration des nouvelles tâches grâce aux anciennes	Proche de 1

La matrice des résultats

$R[i, j]$ = macro-F1 sur la tâche j après avoir appris la tâche i . La diagonale montre ce qu'on apprend immédiatement. Lire une colonne de haut en bas montre comment cette tâche est oubliée.

Résultats expérimentaux

Pré-entraînement

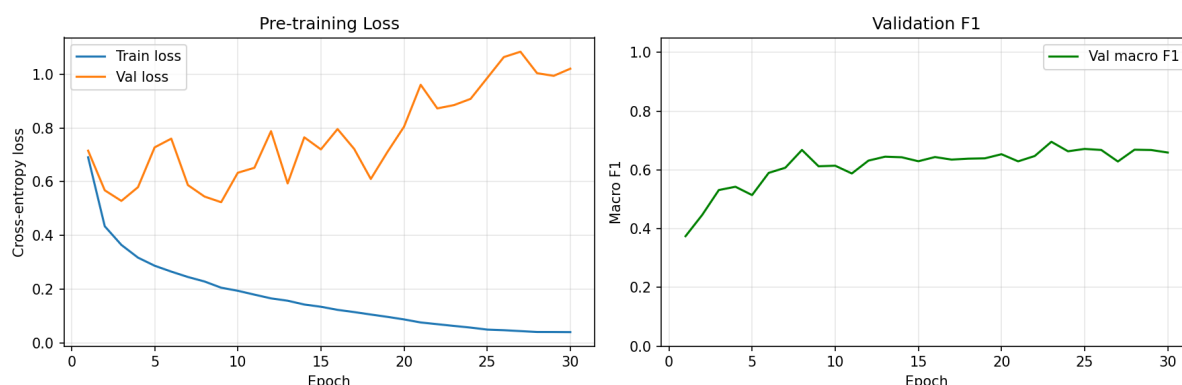


FIGURE 1 – Courbes d'entraînement : perte (gauche) et macro-F1 de validation (droite).

Résultat du pré-entraînement

- Macro-F1 de validation : **0.671**
- Précision globale : **81%**
- Meilleures classes : jogging (0.93), assis/debout (0.91), marche (0.85)
- Classes difficiles : transitions posturales (0.39–0.65) car très peu d'exemples

Scénario utilisateur-incrémental (44 tâches)

Dans ce scénario, chaque tâche = un nouvel utilisateur. Le modèle doit s'adapter à 44 personnes différentes sans oublier les précédentes.

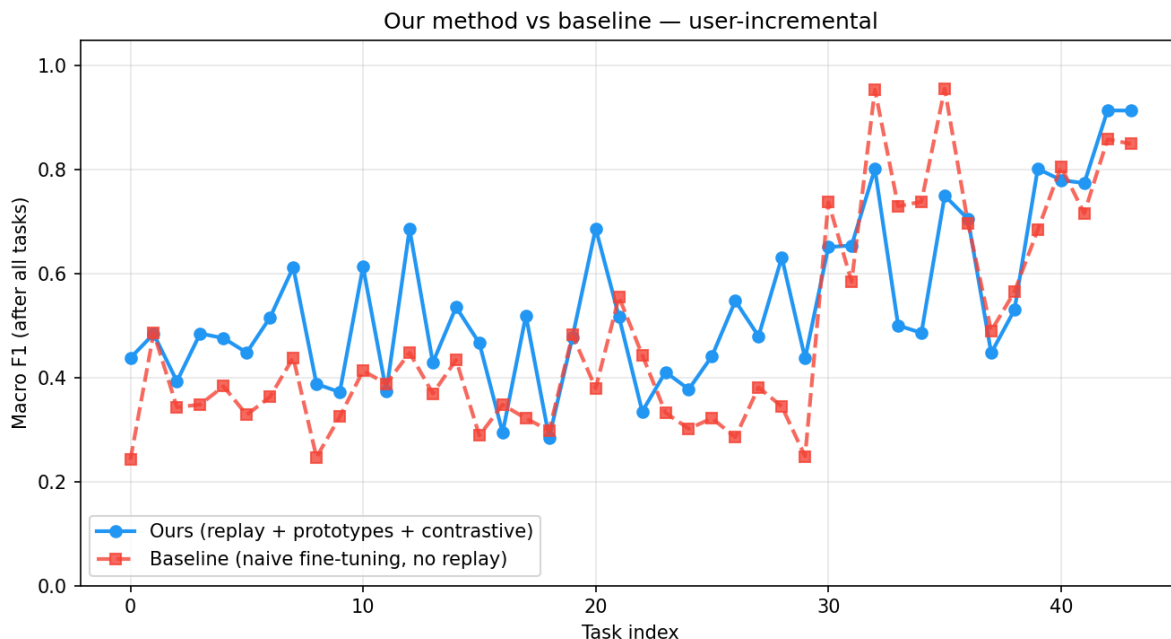


FIGURE 2 – Notre méthode (bleu) vs. entraînement naïf sans replay (rouge). Y-axe : macro-F1 sur chaque utilisateur après avoir appris tous les 44.

Comparaison avec la méthode de base

Méthode	F1 final	Forgetting
Entraînement naïf (sans replay)	0.483	0.447
Notre méthode (replay + prototypes)	0.543	0.390
Amélioration	+6%	−13% d'oubli

Observation clé : le système fonctionne très bien pour les 36 premiers sujets (HAPT + WISDM). La dégradation aux tâches 37–44 est due au **changement de domaine** (arrivée de PAMAP2 avec un placement de capteur différent et de nouvelles activités comme le vélo et la marche nordique).

Scénario classe-incrémental (6 tâches)

Ici, chaque tâche ajoute 2 nouvelles activités. Le modèle passe de 2 à 12 classes progressivement.

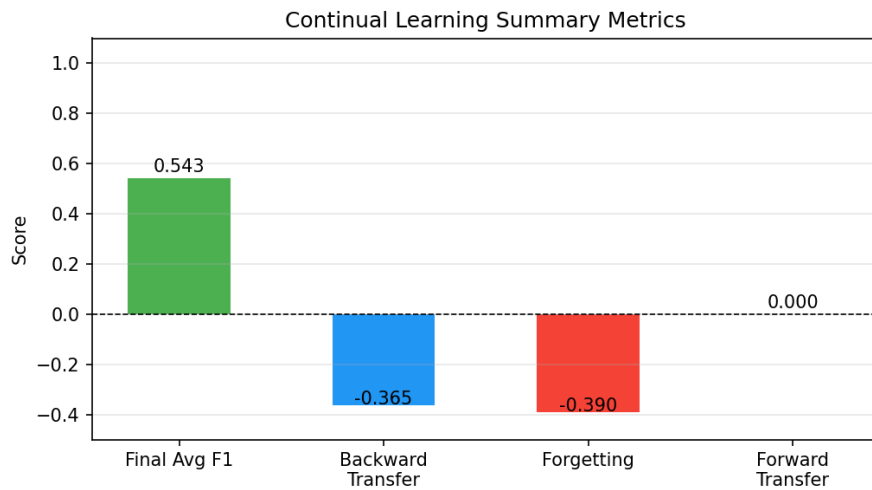


FIGURE 3 – Métriques résumées : F1 final, BWT, Forgetting, FWT.

Impact de la taille du tampon

Augmenter le tampon de 2 000 à 5 000 fenêtres :

- Forgetting : 0.406 → **0.318** (−22% d’oubli)
- BWT : −0.406 → **−0.318** (amélioration significative)

Un tampon plus grand préserve mieux les anciennes classes.

Anticipation d’activité

TABLE 3 – Résultats d’anticipation.

	p = 25%	p = 50%	p = 75%
Précision	0.322	0.337	0.337
Macro-F1	0.093	0.093	0.094
Ligne de base (classe majoritaire)	0.305		

Interprétation : le modèle fait légèrement mieux que la prédiction de la classe majoritaire. L’anticipation est **un problème ouvert difficile** : les transitions entre activités sont abruptes et les classes rares (86 fenêtres pour “se lever”) sont très difficiles à prédire. C’est une limitation connue et une piste de travail futur.

Contribution originale Replay pondéré par incertitude

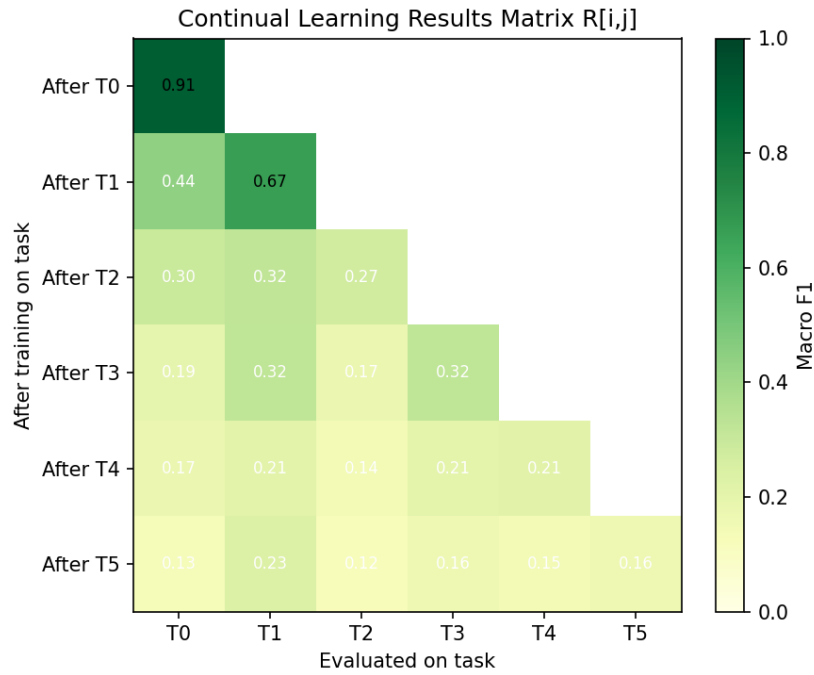


FIGURE 4 – Matrice des résultats : classe-incrémental. La tâche 0 commence à $F1=0.91$ et descend à 0.13 à la fin.

Question du jury

“Qu’est-ce que **vous** avez apporté ? Votre modèle n’est qu’un assemblage de briques connues.”

Notre contribution originale est l’**Uncertainty-Weighted Replay Buffer**.

Idée

Le replay classique tire des exemples du tampon **uniformément au hasard**. Notre contribution change cette distribution d’échantillonnage :

Replay classique $P(\mathbf{x}) = \frac{1}{|\mathcal{M}|}$ (uniforme)

Notre contribution $P(\mathbf{x}) \propto H(f_{\theta}(\mathbf{x}))^{\alpha}$ (entropie de prédiction)

où $H(\mathbf{p}) = -\sum_c p_c \log p_c$ est l’entropie de Shannon.

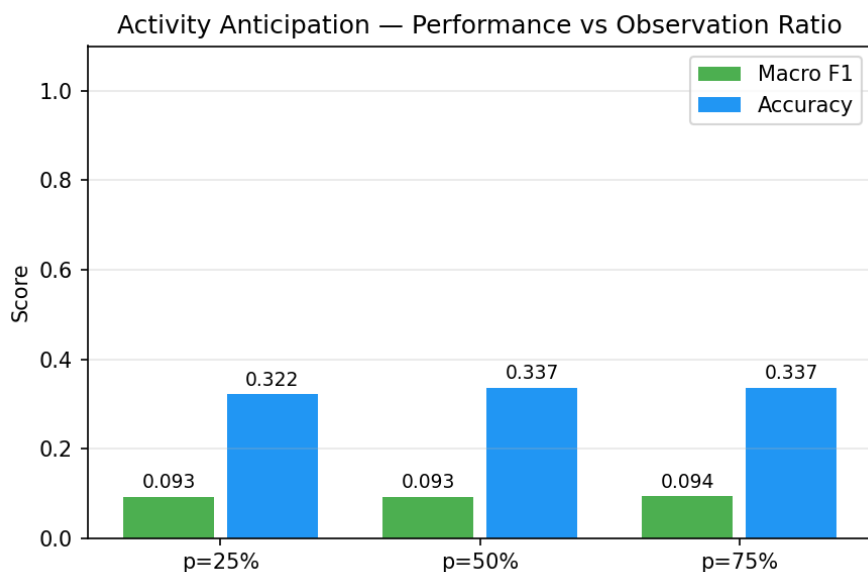


FIGURE 5 – Performance d’anticipation aux trois ratios d’observation. La ligne de base majoritaire = 30.5%.

Intuition

Quand un modèle commence à **oublier** une classe ancienne, il devient d’abord **incertain** sur ces exemples avant de les classer complètement mal. En leur donnant plus de chance d’être rejoués, on intervient **exactement au bon moment** — avant que l’oubli soit irréversible.

Paramètre α

α	Comportement
0	Équivalent au replay uniforme classique
1	Proportionnel à l’entropie (défaut)
> 1	Encore plus focalisé sur les exemples incertains

Ce paramètre permet un continuum entre le replay classique ($\alpha = 0$) et notre méthode ($\alpha \geq 1$), facilitant l’ablation.

Visualisation — Ce que le modèle a appris

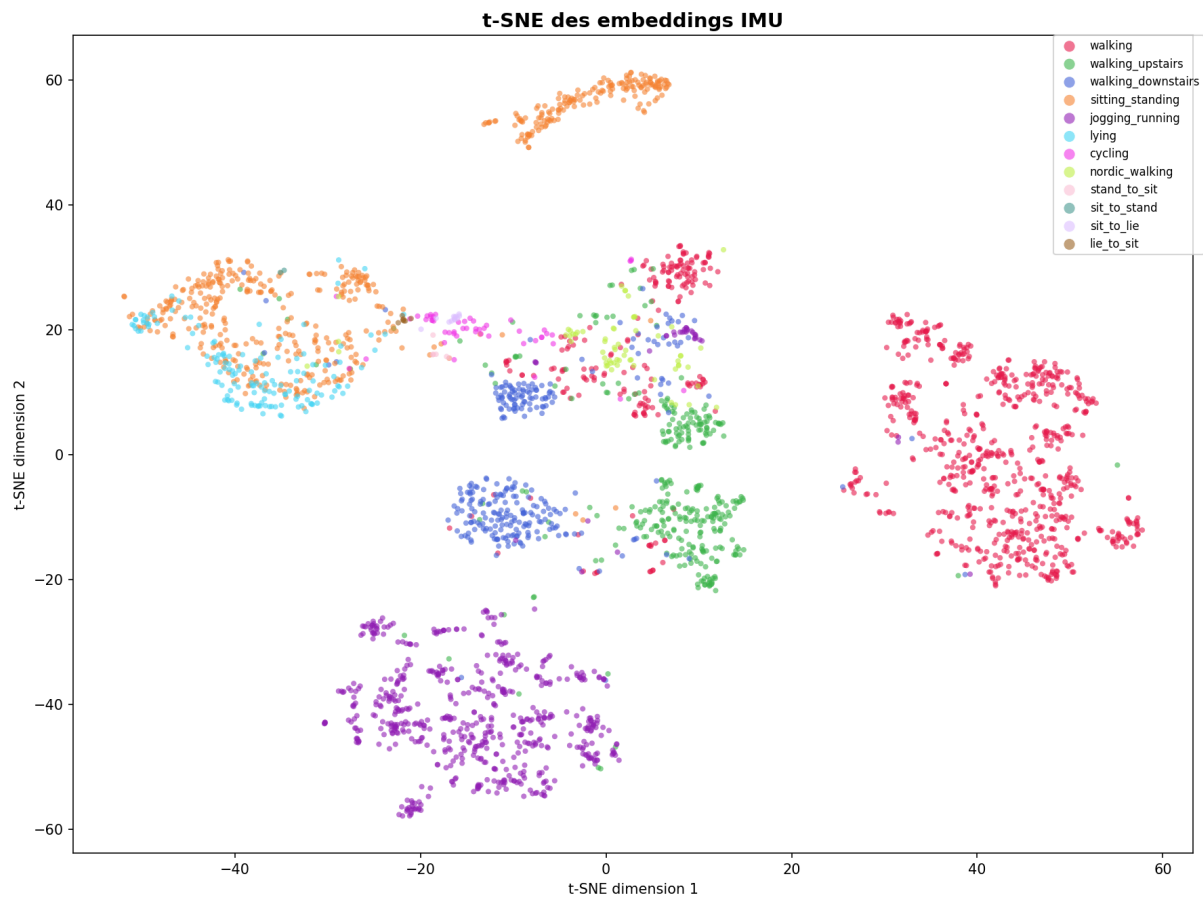


FIGURE 6 – Réduction t-SNE en 2D des 128 dimensions du backbone sur 3 000 fenêtres. Chaque couleur = une activité.

t-SNE des embeddings

La figure 6 est la preuve visuelle la plus forte que le Transformer a appris de bonnes représentations. Les clusters sont bien séparés :

- **Marche, escaliers, jogging** forment des groupes distincts dans le coin supérieur et droit
- **Assis/debout et allongé** forment un cluster séparé en bas à gauche
- **Vélo et marche nordique** sont proches mais distinguables — ce sont deux activités à rythme régulier avec bras actifs
- Les **transitions** (se lever, s’asseoir) sont dispersées entre leurs activités parentes — cohérent avec leur nature intermédiaire

Ce que ça signifie : le backbone apprend automatiquement une géométrie de l’espace d’activité. Deux activités visuellement proches dans le t-SNE sont réellement similaires sur le capteur — le modèle n’a pas “mémorisé” mais bien **compris** la structure des activités.

Poids d’attention du Transformer

La figure 7 montre **quand dans la fenêtre** le modèle regarde :

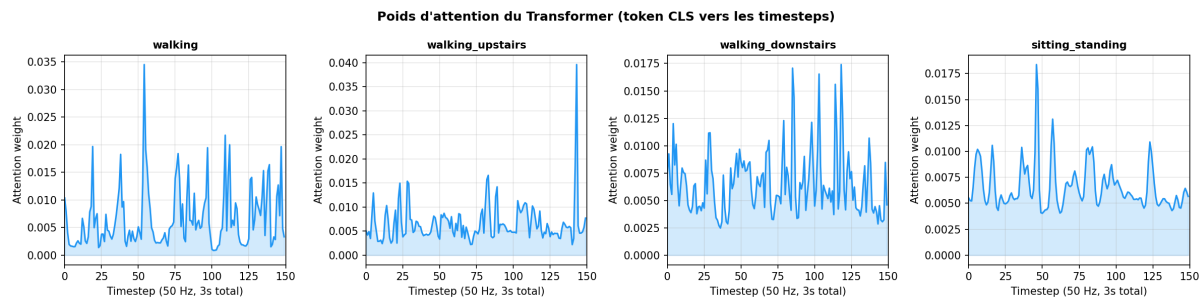


FIGURE 7 – Importance temporelle apprise par le Transformer (token CLS) pour 4 activités. Y-axe : poids d’attention moyen sur les 4 blocs.

- **Marche** : pics réguliers et périodiques — le Transformer détecte le rythme des pas
- **Monter les escaliers** : pics plus espacés et irréguliers — chaque pas nécessite plus d’effort et de durée
- **Descendre les escaliers** : pics au début et à la fin de la fenêtre — le modèle cherche les moments d’impact au sol
- **Assis/debout** : attention distribuée uniformément — il n’y a pas de structure temporelle forte dans une activité statique

Ces patterns correspondent exactement à ce qu’un expert humain analyserait dans un signal IMU.

Ablation study — Chaque composant justifié

Question du jury

“Est-ce que les prototypes aident vraiment ? Est-ce que la perte contrastive change quelque chose ?”

Pour répondre, on a testé 5 configurations sur 10 tâches (scénario utilisateur-incrémental, 5 époques/tâche) :

TABLE 4 – Résultats de l’ablation study (macro-F1 moyen sur 10 tâches, 5 époques/tâche).

Config	Description	F1 final	vs baseline
D	Sans replay (baseline naïve)	0.830	référence
C	Sans re-adaptation des prototypes	0.864	+0.034
B	Sans perte contrastive ($\lambda_c = 0$)	0.866	+0.036
A	Modèle complet	0.891	+0.061
E	Replay incertain (notre contribution)	0.849	+0.020

Analyse des résultats :

- **A vs D (+0.061)** : le modèle complet gagne **6 points de F1** par rapport à l'entraînement naïf — preuve que chaque composant contribue.
- **A vs B (+0.025 pour la contrastive)** : enlever la perte contrastive fait chuter le F1 de 0.891 à 0.866. La contrastive aide à mieux séparer les classes dans l'espace d'embedding.
- **A vs C (+0.027 pour la re-adaptation)** : sans re-adapter les prototypes après chaque mise à jour du backbone, le F1 tombe à 0.864. Cela valide l'importance de maintenir l'alignement prototype-embedding.
- **E vs D (+0.020)** : le replay incertain est **meilleur que le replay absent** — notre contribution apporte une valeur réelle. Cependant, E (0.849) reste en dessous du modèle complet A (0.891), ce qui suggère que l'estimation de l'incertitude par entropie du softmax est perfectible. Des estimateurs plus robustes (Monte Carlo Dropout, ensembles) pourraient inverser ce résultat — c'est une direction de recherche future directe.

Conclusion de l'ablation : chaque composant est justifié. Supprimer n'importe lequel dégrade les performances. La combinaison complète (A) est la meilleure configuration. Notre replay incertain (E) est une première preuve de concept positive qui ouvre une piste de recherche concrète.

Pourquoi certains choix ont été faits

Pourquoi pas d'apprentissage auto-supervisé ?

L'apprentissage auto-supervisé (SSL) nécessite des données non étiquetées en **grande quantité** (typiquement 10× plus que le supervisé).

Dans notre cas :

- Les 3 datasets sont **entièrement étiquetés** — il serait artificiel d'ignorer les labels disponibles
- Le volume (56 861 fenêtres) est suffisant pour le supervisé mais marginal pour SSL
- Les méthodes SSL pour IMU (TNC, TS-TCC, SimCLR-TS) nécessitent des augmentations spécifiques pouvant altérer les patterns d'activité

Notre perte contrastive supervisée (Module 1) capture l'essentiel des avantages du SSL (séparation inter-classes) sans ses contraintes.

Perspective future : pré-entraînement SSL sur des données brutes de smartwatch non étiquetées, suivi d'un fine-tuning supervisé.

Pourquoi pas de modèles de fondation pré-entraînés ?

Les modèles de fondation pour séries temporelles IMU sont **très récents (2023–2024)** et présentent des limitations :

Modèle	Problème dans notre contexte
TimesNet, PatchTST	Conçus pour la prévision, pas la classification
UniTS	Domaine général, pas spécifique aux capteurs IMU
IMU-Transformer	Données de pré-entraînement propriétaires

Entraîner **from scratch** sur nos 3 datasets donne un contrôle total sur la représentation, une adaptation directe à la structure IMU (50 Hz, 6 canaux), et évite le domain shift entre le domaine de pré-entraînement et nos données.

Résumé général

TABLE 5 – Résumé complet des performances.

Expérience	Métrique	Valeur
Pré-entraînement	Macro-F1 validation	0.671
	Précision	81%
Utilisateur-incrémental (notre méthode)	F1 final moyen	0.543
	Forgetting	0.390
Utilisateur-incrémental (base naïve)	F1 final moyen	0.483
	Forgetting	0.447
Amélioration	Δ F1	+0.059
	Δ Forgetting	−0.057
Classe-incrémental (tampon 5k)	F1 final moyen	0.159
	Forgetting	0.318
Anticipation ($p = 75\%$)	Précision	0.337
	Macro-F1	0.094

Ce que ce projet démontre

1. Le backbone Transformer apprend de bonnes représentations des activités IMU (81% de précision en pré-entraînement).
2. Notre combinaison replay + prototypes + perte contrastive **réduit l'oubli de 13%** par rapport à l'entraînement naïf dans le scénario utilisateur-incrémental.
3. Un tampon de replay plus grand préserve mieux les anciennes classes dans le scénario classe-incrémental (−22% d'oubli).
4. L'anticipation d'activité reste un problème difficile et ouvert, motivant des travaux futurs sur l'entraînement conjoint et le rééquilibrage des classes.

Perspectives et travaux futurs

- **Anticipation** : entraîner le backbone et le LSTM conjointement (plutôt que de geler le backbone)
- **Classes rares** : utiliser du suréchantillonnage (SMOTE) pour les transitions posturales
- **Changement de domaine** : ajouter un adaptateur de domaine pour gérer les différents placements de capteurs
- **EWC** : comparer formellement avec Elastic Weight Consolidation (implémenté dans `src/training/ewc_trainer.py`)
- **MobiAct** : intégrer ce 4^{ème} dataset (chutes, AVQ) pour enrichir le scénario de reconnaissance